

CSCE 641 COMPUTER GRAPHICS · MODULE 11 · LECTURE NOTES

Physically Based Shading and Image-Based Lighting

Replacing the curve fit with something that conserves energy.

MODULE OUTCOMES

State the reflectance equation and identify each term in a shader.

Explain the microfacet model and the role of D, F, and G.

Implement a Cook–Torrance BRDF with metallic/roughness parameters.

Explain the split-sum approximation and implement image-based lighting.

Explain why tonemapping is required once lighting is physical.

1 The reflectance equation

Module 06 built shading from plausible curves. This module starts from the physics and derives what the curve should have been. The starting point is the reflectance equation, which says how much light leaves a surface in a given direction:

$$L_o(\mathbf{v}) = \int_{\Omega} f_r(\mathbf{l}, \mathbf{v}) \cdot L_i(\mathbf{l}) \cdot (\mathbf{n} \cdot \mathbf{l}) \, d\omega$$

Term	Meaning
$L_o(\mathbf{v})$	Radiance leaving toward the viewer — what the pixel measures.
$\int_{\Omega}$	Integrate over the hemisphere above the surface: light arrives from every direction, not just from a list of lights.
$f_r(\mathbf{l}, \mathbf{v})$	The BRDF. The complete description of the material, and the only term you get to choose.
$L_i(\mathbf{l})$	Radiance arriving from direction $\mathbf{l}$.
$(\mathbf{n} \cdot \mathbf{l})$	Module 06's cosine law: the geometric foreshortening of the incoming beam.

Module 06's diffuse shader is this integral in a degenerate case: a constant BRDF and a single delta-function light direction, so the integral collapses to a product. Everything in this module is about doing better on the BRDF, and then about evaluating the integral when the light is an entire environment rather than a point.

1.1 What a BRDF must satisfy

- 1 **Positivity.** $f_r \geq 0$ everywhere. Surfaces do not remove light.
- 2 **Helmholtz reciprocity.** $f_r(l, v) = f_r(v, l)$. Exchanging the light and the eye gives the same result. This is required for bidirectional path tracing to be valid at all.
- 3 **Energy conservation.** For any incoming direction, the integral of the BRDF times the cosine over the outgoing hemisphere is at most 1. A surface cannot reflect more light than it receives.

WHY ENERGY CONSERVATION IS THE PRACTICAL ONE

Blinn–Phong violates it: changing the shininess exponent changes total reflected energy arbitrarily.

The production consequence is that materials do not transfer. A material tuned to look right in one lighting setup looks wrong in another, so every asset must be re-authored per environment. That cost, multiplied across a large project, is enormous.

An energy-conserving model with physically meaningful parameters makes materials portable. That, far more than any single image being prettier, is why the industry moved to PBR.

2 Microfacet theory

Model the surface as a vast collection of microscopic perfect mirrors — **microfacets** — each far too small to resolve. What we call roughness is the statistical distribution of their orientations: a smooth surface has facets mostly aligned with the macroscopic normal, a rough one has them scattered widely.

A perfect mirror reflects light from $\mathbf{l}$ to $\mathbf{v}$ only if its normal is exactly the halfway vector $\mathbf{h} = \text{normalize}(\mathbf{l} + \mathbf{v})$. So the only facets contributing to what you see are those oriented along $\mathbf{h}$. The halfway vector of Blinn–Phong, introduced in Module 06 as a cheaper approximation to the reflection vector, turns out to be the physically meaningful quantity all along.

2.1 The Cook–Torrance form

$$f_{\text{spec}} = D(\mathbf{h}) \cdot F(\mathbf{v}, \mathbf{h}) \cdot G(\mathbf{l}, \mathbf{v}) / (4 (\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v}))$$

Term	Models	Controls	Fixes which Blinn–Phong defect
D — normal distribution	What fraction of facets are oriented along $\mathbf{h}$.	Highlight size and shape.	The arbitrary, unnormalised exponent.
F — Fresnel	How much light reflects rather than refracting into the surface.	Grazing-angle brightening; metal colour.	The missing rim reflection that made everything look like plastic.
G — geometry term	Facets shadowing and masking each other.	Darkening at grazing angles on rough surfaces.	Energy conservation, together with the $4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})$ denominator.

2.2 D: GGX

$$D_{\text{GGX}}(\mathbf{h}) = \alpha^2 / [\pi ((\mathbf{n} \cdot \mathbf{h})^2 (\alpha^2 - 1) + 1)^2]$$

GGX (Trowbridge–Reitz) replaced Beckmann and Phong distributions in production for one reason: its **long tail**. Real measured materials show a tight bright core surrounded by a wide, dim halo, and GGX reproduces that while Gaussian-like distributions cut off too quickly. The difference is most striking on metals, where the halo carries a great deal of the perceived material character.

Note the parameterisation: $\alpha = (\text{perceptual roughness})^2$. Artists author perceptual roughness because the squared mapping distributes visual change more evenly across the slider. Getting this wrong makes the bottom half of the roughness range nearly unusable.

2.3 F: Fresnel

$$F_{\text{Schlick}}(\mathbf{v}, \mathbf{h}) = F_0 + (1 - F_0)(1 - \mathbf{v} \cdot \mathbf{h})^5$$

Every surface becomes more reflective as the viewing angle approaches grazing — look along a sheet of paper and it is nearly a mirror. Schlick's approximation captures this in one line and is accurate enough that essentially nobody uses the full Fresnel equations in real time.

F_0 IS WHERE METALS AND DIELECTRICS PART

Dielectrics (plastic, wood, skin, stone, water): F_0 is about 0.04 and achromatic. Light that is not reflected refracts into the surface, scatters, and emerges as diffuse colour.

Metals (gold, copper, iron): F_0 is high and *coloured* — it is the metal's characteristic colour. Light that is not reflected is absorbed by the free electrons, so there is **no diffuse term at all**.

This is why a gold highlight is gold while a red plastic ball has a white highlight, and it is the single most useful fact in the whole metallic/roughness workflow.

3 Implementation

```
float D_GGX(float NdotH, float a) {
    float a2 = a*a;
    float d = NdotH*NdotH * (a2 - 1.0) + 1.0;
    return a2 / (PI * d * d);
}

vec3 F_Schlick(float VdotH, vec3 F0) {
    return F0 + (1.0 - F0) * pow(1.0 - VdotH, 5.0);
}

// Smith height-correlated visibility: G / (4 NdotL NdotV) in one term.
float V_SmithGGX(float NdotL, float NdotV, float a) {
    float l = NdotV * (NdotL * (1.0 - a) + a);
    float v = NdotL * (NdotV * (1.0 - a) + a);
    return 0.5 / max(l + v, 1e-5);
}

void shade() {
    float a = roughness * roughness;          // perceptual -> alpha
    vec3 F0 = mix(vec3(0.04), albedo, metallic);
    vec3 F = F_Schlick(VdotH, F0);

    vec3 specular = D_GGX(NdotH, a) * V_SmithGGX(NdotL, NdotV, a) * F;

    // Energy bookkeeping: what is not reflected specularly may scatter
    // diffusely -- and metals have no diffuse at all.
    vec3 kD = (1.0 - F) * (1.0 - metallic);
    vec3 diffuse = kD * albedo / PI;           // Module 06's 1/pi

    Lo += (diffuse + specular) * lightColor * NdotL;
}
```

THE LINE PEOPLE DELETE

$kD = (1 - F) * (1 - \text{metallic})$ is what makes the model conserve energy. It says: light reflected specularly is not available to scatter diffusely, and metals absorb rather than scatter.

When a metal 'looks too dark', the instinct is to remove the $(1 - \text{metallic})$ factor. Do not. A dark metal under a single point light is *correct* — metals derive almost all their appearance from reflecting their environment, and the fix is image-based lighting, not breaking the energy balance.

4 Image-based lighting

Real surfaces are lit from every direction by a whole environment, not by two or three point lights. The reflectance equation is an integral over the hemisphere, and with an environment map as L_i it would require thousands of samples per pixel. That is not a real-time budget.

4.1 The split-sum approximation

Karis's observation, from the Unreal Engine 4 course notes, is that the integral factors approximately into two independent pieces:

$$\int \mathbf{f}_r \cdot \mathbf{L}_i(\mathbf{n} \cdot \mathbf{l}) \, d\omega \approx \left[\int \mathbf{L}_i \, d\omega \right] \times \left[\int \mathbf{f}_r(\mathbf{n} \cdot \mathbf{l}) \, d\omega \right]$$

- The **first** factor depends only on the environment and the roughness. Precompute it by convolving the environment map with the GGX lobe at several roughness values, stored in the mip chain of a cube map. Done once per environment.
- The **second** factor depends only on roughness and $\mathbf{n} \cdot \mathbf{v}$, not on the environment at all. It is therefore a scene-independent 2D lookup table, computed once ever and shipped with the engine.

The approximation assumes the view, normal, and reflection directions coincide, which is exact head-on and degrades toward grazing angles. The error shows mainly as slightly wrong stretching of reflections on rough metal at oblique angles, and in practice nobody notices.

```
// Diffuse: irradiance map, cosine-convolved offline
vec3 irradiance = texture(uIrradianceMap, N).rgb;
vec3 diffuseIBL = irradiance * albedo;

// Specular: prefiltered environment + BRDF LUT
vec3 R = reflect(-V, N);
vec3 prefiltered = textureLod(uPrefilteredMap, R,
                             roughness * MAX_REFLECTION_LOD).rgb;
vec2 ab = texture(uBRDFLut, vec2(NdotV, roughness)).rg;
vec3 specularIBL = prefiltered * (F0 * ab.x + ab.y);

vec3 kD = (1.0 - F) * (1.0 - metallic);
vec3 ambient = (kD * diffuseIBL + specularIBL) * occlusion;
```

AMBIENT OCCLUSION MULTIPLIES AMBIENT ONLY

AO represents local self-occlusion of *indirect* light arriving from the environment. Direct light already has a shadow map (Module 10) telling it what is occluded.

Multiplying direct light by AO as well double-darkens creases and makes the image look dirty. It is one of the most common PBR implementation errors, and it is visible once you know to look for it.

5 Tonemapping

Once lighting is physically based, the values are physical. Sunlight is on the order of 100,000 lux; a lit interior is a few hundred. The ratio between the brightest and darkest meaningful values in a scene can easily exceed $10^5:1$, and a display offers roughly 100:1.

Clipping the excess destroys it: every highlight becomes a flat white region, all colour within it is lost, and the image acquires the blown-out look characteristic of early HDR rendering. A **tonemapping** operator instead compresses the range smoothly, preserving contrast where the eye attends and rolling off gracefully

into the highlights.

Operator	Character	Use
Reinhard	Simple, desaturates highlights noticeably.	Easy starting point; fine for learning.
ACES filmic	Film-like roll-off, slightly contrasty, well understood.	The de facto standard for a decade; still a good default.
Khronos PBR Neutral	Preserves material colour into highlights.	Preferred where material fidelity matters, e.g. product visualisation.
AgX	Very graceful hue handling at extreme exposure.	Increasingly common; Blender's current default.

ORDER MATTERS

Render in HDR → tonemap → encode to sRGB (Module 06). In that order, every time.

Tonemapping an already sRGB-encoded image applies a curve to a curve and produces muddy, low-contrast output. Encoding before tonemapping is the same error in the other direction.

6 Project 2

You now have everything the second project requires: the microfacet BRDF from this module, image-based lighting from the split sum, shadow mapping from Module 10, antialiasing from Module 09, and the GPU pipeline from Module 07. Build the material sphere grid — roughness across one axis, metallic across the other — under a real HDR environment. It is the standard validation image for a PBR implementation, and errors in any of the above are immediately visible in it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Brian Karis — 'Real Shading in Unreal Engine 4' (SIGGRAPH course, free)	The split-sum approximation in the author's own words, with the practical shortcuts. The single most important reference for this module.
Sébastien Lagarde & Charles de Rousiers — 'Moving Frostbite to PBR' (free)	The most complete free account of a production PBR pipeline, including all the details other papers omit.
LearnOpenGL — PBR Theory, Lighting, and IBL	Complete working implementation of everything in this module, including generating the irradiance map, prefiltered map, and BRDF LUT.
Naty Hoffman — 'Physics and Math of Shading' (free)	The derivation of microfacet theory from first principles. Read this if the D, F, G decomposition feels arbitrary.

Resource	What it covers for this module
Physically Based Rendering, 4th ed. — free online	Chapter 9 on reflection models. The rigorous treatment, and the bridge to CSCE 647.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the Cook–Torrance BRDF with GGX, Schlick Fresnel, and Smith visibility. Render a 7×7 sphere grid sweeping roughness against metallic under a single point light.
- 2 Verify energy conservation numerically: integrate your BRDF times the cosine over the hemisphere by Monte Carlo sampling, for several roughness values, and confirm the result never exceeds 1. Do the same for Blinn–Phong and record where it fails.
- 3 Generate the three IBL resources yourself from an HDR environment map: the irradiance cube map, the roughness-mipped prefiltered cube map, and the BRDF lookup table. Do not use a precomputed LUT.
- 4 Render the same sphere grid under IBL. Compare the metals against the point-light version and explain in writing why they looked wrong before.
- 5 Implement Reinhard and ACES tonemapping with an exposure control. Capture a high-dynamic-range scene with no tonemapping, with Reinhard, and with ACES.
- 6 Deliberately apply ambient occlusion to direct light as well as ambient. Capture the difference and describe the artifact.
- 7 **Project 2 is now due.** Assemble Modules 07–11 into the GPU renderer described in the syllabus, including the annotated RenderDoc capture.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the reflectance equation and identify which term is the material, which is geometry, and which is the lighting.
- 2 State the three properties a physical BRDF must satisfy. Which does Blinn–Phong violate, and what is the production consequence?
- 3 Explain the microfacet model, and why the halfway vector is the physically meaningful direction.
- 4 What does each of D, F, and G control, and which Blinn–Phong defect does each repair?
- 5 Why do metals have no diffuse term? Why is their F_0 coloured?

- 6 Explain the split-sum approximation: what are the two factors, what does each depend on, and what does the approximation assume?
- 7 Why does physically based lighting force you to tonemap, and in what order do tonemapping and sRGB encoding occur?